

SQL Analytics II

Módulo II



Temario

- Comandos de manipulación y definición
 - SQL - Lenguaje de Definición de Datos (DDL)
 - Create
 - Drop
 - Alter
 - Truncate
 - SQL – Lenguaje de Manipulación de Datos (DML)
 - Insert
 - Update
 - Delete

SQL > Lenguaje de Definición de Datos

DDL (Data Definition Language)

- Es un Lenguaje que permite modificar la estructura de las tablas de una base de datos
- Permite por ejemplo:
 - Crear una tabla
 - Alterar los campos de una tabla
 - Borrar una tabla
 - Vaciar el contenido de una tabla
 - Renombrar una tabla

SQL

DDL

CREATE

- CREATE TABLE

- La declaración CREATE nos permite crear tablas, vistas, índices y bases de datos.
- Durante el curso nos centraremos en crear tablas temporales por lo que agregaremos la palabra TABLE (tabla).

SQL

DDL

CREATE

- CREATE TABLE
- `CREATE TABLE #[nombre_tabla] ([Nombre Campo 1] [tipo de dato],
[Nombre Campo 2] [tipo de dato],[Nombre Campo n][tipo de dato])`

Al crear una tabla, definimos su nombre y para cada campo tenemos que especificar el tipo de dato albergara el campo

SQL

DDL

CREATE

- CREATE TABLE

```
CREATE TABLE #Employee (  
    EmployeeID INT,  
    FirstName NVARCHAR(50),  
    LastName NVARCHAR(50),  
    EMail NVARCHAR(50),  
    Phone VARCHAR(15),  
    HireDate DATE,  
    Salary MONEY);
```

Creamos una tabla temporal llamada #Employee, definiendo sus campos y el tip de dato para cada campo

SQL

DDL

DROP

- DROP TABLE

- Quita una o más definiciones de tabla y todos los datos, índices, desencadenadores, restricciones y especificaciones de permisos para esas tablas.
- Quita una o más definiciones de tabla y todos los datos, índices, desencadenadores, restricciones y especificaciones de permisos para esas tablas.

SQL

DDL

DROP

- DROP TABLE
- DROP TABLE #[Nombre_de_la_tabla]

Para eliminar una tabla especificamos su nombre. Si la tabla que intentamos eliminar no existe, el sistema nos dará un mensaje de error

SQL

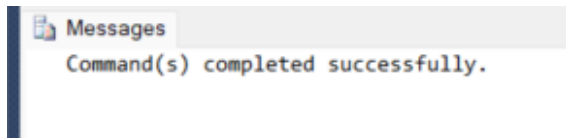
DDL

DROP

- CREATE TABLE

Cuando la tabla existe

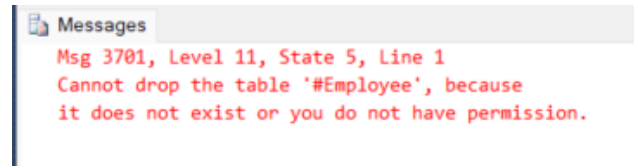
DROP TABLE #Employee



Cuando eliminamos una tabla existente el sistema nos confirma su eliminación.

Cuando la tabla no existe

DROP TABLE #Employee



Cuando eliminamos una tabla que no existe el sistema nos devuelve un mensaje error.

SQL

DDL

ALTER

- ALTER TABLE
 - Modifica una definición de tabla alterando, agregando (ADD) o eliminando (DROP) columnas y restricciones, reasignando particiones, o deshabilitar o habilitar restricciones y desencadenadores.

SQL

DDL

ALTER

- ALTER TABLE ADD

```
ALTER TABLE #[Nombre_de_la_tabla]  
ADD [Nombre Campo 1] [tipo de dato]
```

Para agregar una nueva columna, primero seleccionamos la tabla con **ALTER TABLE**, y luego escribimos **ADD**, el nombre de la nueva columna(s) y su(s) tipo de datos

SQL

DDL

ALTER

- ALTER TABLE DROP

ALTER TABLE #[Nombre_de_la_tabla]
DROP [Nombre Campo 1] [tipo de dato]

Para eliminar una columna o mas columnas, primero seleccionamos la tabla con **ALTER TABLE**, y luego escribimos **DROP COLUMN** y el nombre de la(s) columna(s) a eliminar

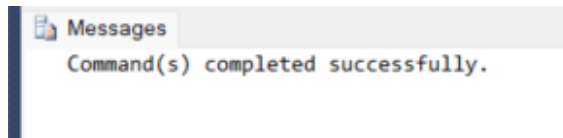
SQL

DDL

ALTER

- ALTER TABLE

```
ALTER TABLE #Employee  
ADD MaritalStatus NVARCHAR(1)
```



En este ejemplo agregamos una nueva columna llamada MaritalStatus a la tabla #Employee

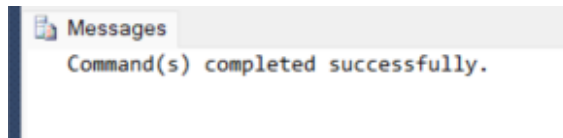
SQL

DDL

ALTER

- ALTER TABLE

```
ALTER TABLE #Employee  
DROP COLUMN MaritalStatus
```



En este ejemplo eliminamos la columna llamada MaritalStatus a la tabla #Employee.

SQL

DDL

TRUNCATE

- TRUNCATE TABLE

- Quita todas las filas de una tabla sin eliminar la propia tabla.
- TRUNCATE TABLE es similar a la declaración DELETE sin cláusula WHERE; sin embargo, TRUNCATE TABLE es más rápido y utiliza menos recursos del sistema y del registro de transacciones.

SQL

DDL

TRUNCATE

- TRUNCATE TABLE

TRUNCATE TABLE #[Nombre_de_la_tabla]

Para eliminar todos los registros de una tabla utilizamos la sentencia **TRUNCATE TABLE** y luego especificamos el nombre de la tabla a la que le eliminaremos todos los registros.

SQL

DDL

TRUNCATE

- TRUNCATE TABLE

TRUNCATE TABLE #Employee

Eliminamos todos los registros de la tabla #Employee.

SQL

Lenguaje de Manipulación de Datos

DML (Data Manipulation Lenguaje)

- Es un Lenguaje que permite la manipulación de los datos
- Permite por ejemplo:
 - Consultar registros
 - Insertar registros en la Base de Datos
 - Actualizar registros
 - Borrar registros

SQL

DML

INSERT

- INSERT INTO

- La sentencia INSERT INTO nos permite insertar datos en una base de datos.
- Podemos insertar una fila que indique la información de cada columna existente (en orden),
- Podemos insertar una fila especificando las columnas que desamos completar. Se pueden insertar registros con algunos campos y los no mencionados quedan nulos

SQL

DML

INSERT

- INSERT INTO sin especificar campos

```
INSERT INTO #[Nombre_de_la_tabla]  
VALUES ([Valor Campo 1],[Valor Campo 2],...,[Valor Campo n])
```

- Esta sintaxis tiene las siguientes ventajas y desventajas:
 - Nos obliga a definir los valores de todos los campos, respetando el orden de las columnas.
 - El orden de las campos debe seguir siendo el mismo, de lo contrario, algunos valores corren el riesgo de asignarse al campo incorrecto.

SQL

DML

INSERT

- INSERT INTO sin especificar campos

```
INSERT INTO #Employee  
VALUES (123456, 'Eufemio', 'Masculino', 'eufemio@gmail.com', '254-25874-  
5623', '2019-05-23', 58475, 'S')
```

```
SELECT * FROM #Employee  
WHERE EmployeeID = 123456
```

EmployeeID	FirstName	LastName	EMail	Phone	HireDate	Salary	MaritalStatus
123456	Eufemio	Masculino	eufemio@gmail.com	254-25874-5623	2019-05-23	58475,00	S

En este ejemplo agregamos un registro para un nuevo empleado, y detallamos los valores de todos los campos en la tabla

SQL

DML

INSERT

- INSERT INTO especificando campos

INSERT INTO #[Nombre_de_la_tabla] ([Nombre Campo 1],[Nombre Campo 2],[Valor Campo 5]
VALUES ([Valor Campo 1],[Valor Campo 2],[Valor Campo 5])

- Esta sintaxis tiene las siguientes ventajas y desventajas:
 - Nos obliga a definir los nombres de los campos a ser modificados, antes de “**VALUES**”
 - Es posible asignar valores a ciertos campos y dejar los restantes como nulos o con valores por defecto.

SQL

DML

INSERT

- INSERT INTO especificando campos

```
INSERT INTO #Employee (EmployeeID, FirstName, LastName, EMail, HireDate, Salary)
```

```
VALUES (125845, 'Maria', 'King', 'mariak@gmail.com', '2020-02-12', 65214)
```

```
SELECT * FROM #Employee  
WHERE EmployeeID = 125845
```

EmployeeID	FirstName	LastName	EMail	Phone	HireDate	Salary	MaritalStatus
125845	Maria	King	mariak@gmail.com	NULL	2020-02-12	65214,00	NULL

En este ejemplo al haber especificado solo algunos de los campos, los no referenciados quedan nulos en la base.

SQL

DML

INSERT

- INSERT INTO especificando campos

```
INSERT INTO #[Nombre_de_la_tabla] ([Nombre Campo 1],[Nombre  
Campo 2],[Valor Campo 5]  
VALUES ([Valor Campo 1],[Valor Campo 2],[Valor Campo 5])
```

- Esta sintaxis tiene las siguientes ventajas y desventajas:
 - Nos obliga a definir los nombres de los campos a ser modificados, antes de “VALUES”
 - Es posible asignar valores a ciertos campos y dejar los restantes como nulos o con valores por defecto.

SQL

DML

INSERT

- INSERT INTO varios registros a la vez

INSERT INTO #[Nombre_de_la_tabla] ([Nombre Campo 1],[Nombre Campo 2],...,[Valor Campo n]

VALUES ([Valor Campo 1],[Valor Campo 2],[Valor Campo n])
([Valor Campo 1],[Valor Campo 2],[Valor Campo n])
([Valor Campo 1],[Valor Campo 2],[Valor Campo n])
([Valor Campo 1],[Valor Campo 2],[Valor Campo n])

- Obsérvese que la información de cada nueva tupla o registro esta contenida entre paréntesis curvos

SQL

DML

INSERT

- INSERT INTO varios registros a la vez

INSERT INTO #Employee
VALUES

(123658,'Juana','Kochhar','juanak@gmail.com','606.654.8654','2021-10-12',51000,'S'),
(124692,'Miguel','De Haan','miguelhd@hotmail.com','548-123-4569','2022-05-25',16000,'S');

SELECT * FROM #Employee WHERE EmployeeID IN (123658, 124692)

EmployeeID	FirstName	LastName	EMail	Phone	HireDate	Salary	MaritalStatus
123658	Juana	Kochhar	juanak@gmail.com	606.654.8654	2021-10-12	51000,00	S
124692	Miguel	De Haan	miguelhd@hotmail.com	548-123-4569	2022-05-25	16000,00	S

En este ejemplo se insertaron dos registros en la misma sentencia. Se puede realizar especificando solo algunos campos, siempre que sean mencionados antes de **VALUE**,

SQL

DML

UPDATE

- UPDATE

- La sentencia UPDATE nos permite realizar modificaciones en registros existentes.
- Se suele utilizar junto con el comando WHERE para especificar que filas se actualizaran.
- Podemos modificar uno o mas campos a la vez.

SQL

DML

UPDATE

- UPDATE

```
UPDATE #[Nombre_de_la_tabla]
SET [Nombre Campo 1] = [Nuevo Valor Campo 1],
    [Nombre Campo 2] = [Nuevo Valor Campo 2]
WHERE [Condición]
```

La clausula WHERE nos permite modificar aquellos registros que cumplen la condición, de no especificar una condición todos los registros de la tabla sufrirían la modificación

SQL

DML

UPDATE

- UPDATE

SELECT * **FROM** #Employee **WHERE** EmployeeID = 125845;

EmployeeID	FirstName	LastName	EMail	Phone	HireDate	Salary	MaritalStatus
125845	Maria	King	mariak@gmail.com	NULL	2020-02-12	65214,00	NULL

UPDATE #Employee

SET Phone = '251-369-1258',

MaritalStatus = 'S'

WHERE EmployeeID = 125845;

SELECT * **FROM** #Employee **WHERE** EmployeeID = 125845;

EmployeeID	FirstName	LastName	EMail	Phone	HireDate	Salary	MaritalStatus
125845	Maria	King	mariak@gmail.com	251-369-1258	2020-02-12	65214,00	S

En este ejemplo actualizamos el teléfono y el estado civil del empleado 125845

SQL

DML

DELETE

- DELETE

- La sentencia UPDATE nos permite eliminar registros de una tabla.
- Se suele utilizar junto con el comando WHERE para especificar que filas se eliminarán,
- Según la condición especificada se pueden eliminar 1 o más registros.

SQL

DML

DELETE

- DELETE

```
DELETE FROM #[Nombre_de_la_tabla]  
WHERE [Condición]
```

La clausula WHERE nos permite eliminar los regusotrs que cumplen la condición.

Cuidado, si no se especifica una condición, todos los registros de la tabla serán eliminados.

SQL

DML

DELETE

- DELETE

```
SELECT EmployeeID, FirstName, LastName  
FROM #Employee;
```

EmployeeID	FirstName	LastName
123456	Eufemio	Masculino
125845	Maria	King
123658	Juana	Kochhar
124692	Miguel	De Haan

```
DELETE FROM #Employee  
WHERE EmployeeID= 123456;
```

```
SELECT EmployeeID, FirstName, LastName  
FROM #Employee;
```

EmployeeID	FirstName	LastName
125845	Maria	King
123658	Juana	Kochhar
124692	Miguel	De Haan

En este ejemplo eliminamos el registro perteneciente al empleado con ID 123456.

Resumen Módulo I

- Comandos de manipulación y definición
 - SQL - Lenguaje de Definición de Datos (DDL)
 - Create
 - Drop
 - Alter
 - Truncate
 - SQL – Lenguaje de Manipulación de Datos (DML)
 - Insert
 - Update
 - Delete